

Handling Missing Values

```
import pandas as pd
import numpy as np

nfl_data = pd.read_csv("../input/nflplaybyplay2009to2016/NFL Play by Play
2009-2017 (v4).csv")

np.random.seed(0)
```

```
nfl_data.head()
```

	Date	GameID	Drive	qtr	down	time	TimeUnder	TimeSecs	PlayTimeDiff	SideoffField	...	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yackWPA
0	2009-09-10	2009091000	1	1	NaN	15:00	15	3600.0	0.0	TEN	...	NaN	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	NaN	NaN
1	2009-09-10	2009091000	1	1	1.0	14:53	15	3593.0	7.0	PIT	...	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036889
2	2009-09-10	2009091000	1	1	2.0	14:16	15	3556.0	37.0	PIT	...	NaN	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	NaN	NaN
3	2009-09-10	2009091000	1	1	3.0	13:35	14	3515.0	41.0	PIT	...	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156233
4	2009-09-10	2009091000	1	1	4.0	13:27	14	3507.0	8.0	PIT	...	NaN	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	NaN	NaN

```
missing_values_count = nfl_data.isnull().sum()
Missing_values_count[0:10]
```

	0
Date	0
GameID	0
Drive	0
qtr	0
down	23030
time	83
TimeUnder	0
TimeSecs	83
PlayTimeDiff	166
SideoffField	199

```
total_cells = np.product(nfl_data.shape)
total_missing = missing_values_count.sum()
percent_missing = (total_missing/total_cells) * 100
print(percent_missing)
```

```
27.702828571213928
```

```
missing_values_count[0:10]
```

	0
Date	0
GameID	0
Drive	0
qtr	0
down	23030
time	83
TimeUnder	0
TimeSecs	83
PlayTimeDiff	166
SideofField	199

```
nfl_data.dropna()
```

```
Date GameID Drive qtr down time TimeUnder TimeSecs PlayTimeDiff SideofField ... yacEPA Home_WP_pre Away_WP_pre Home_WP_post Away_WP_post Win_Prob WPA airWPA yachWPA Season
```

```
columns_with_na_dropped = nfl_data.dropna(axis=1)
```

```
columns_with_na_dropped.head()
```

	Date	GameID	Drive	qtr	TimeUnder	ydstogo	ydsnet	PlayAttempted	Yards.Gained	sp	...	Accepted.Penalty	Penalty.Yards	HomeTeam	AwayTeam	Timeout_Indicator	posteam_timeouts_pre	HomeTime
0	2009-09-10	2009091000	1	1	15	0	0	1	39	0	...	0	0	PIT	TEN	0	3	
1	2009-09-10	2009091000	1	1	15	10	5	1	5	0	...	0	0	PIT	TEN	0	3	
2	2009-09-10	2009091000	1	1	15	5	2	1	-3	0	...	0	0	PIT	TEN	0	3	
3	2009-09-10	2009091000	1	1	14	8	2	1	0	0	...	0	0	PIT	TEN	0	3	
4	2009-09-10	2009091000	1	1	14	8	2	1	0	0	...	0	0	PIT	TEN	0	3	

```
print("Columns in original dataset: %d \n" %
```

```
nfl_data.shape[1])
```

```
print("Columns with na's dropped: %d" %
```

```
columns_with_na_dropped.shape[1])
```

```
Columns in original dataset: 102
```

```
Columns with na's dropped: 34
```

```
subset_nfl_data = nfl_data.loc[:, 'EPA':'Season'].head()
```

```
subset_nfl_data
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	NaN	NaN	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	NaN	NaN	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	NaN	NaN	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	NaN	NaN	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	NaN	NaN	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	NaN	NaN	2009.0

```
subset_nfl_data.fillna(0)
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	-1.068169	1.146076	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	-0.032244	0.036899	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	3.318841	-5.031425	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	0.106663	-0.156239	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	0.000000	0.000000	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	0.000000	0.000000	2009.0

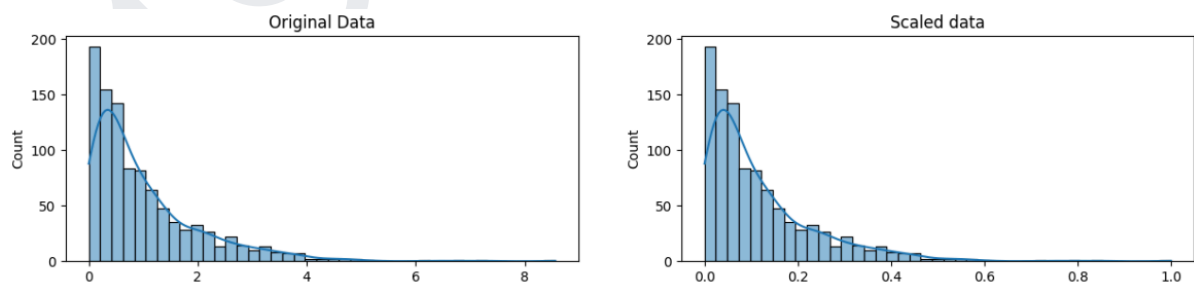
```
subset_nfl_data.fillna(method='bfill', axis=0).fillna(0)
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	-1.068169	1.146076	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	-0.032244	0.036899	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	3.318841	-5.031425	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	0.106663	-0.156239	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	0.000000	0.000000	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	0.000000	0.000000	2009.0

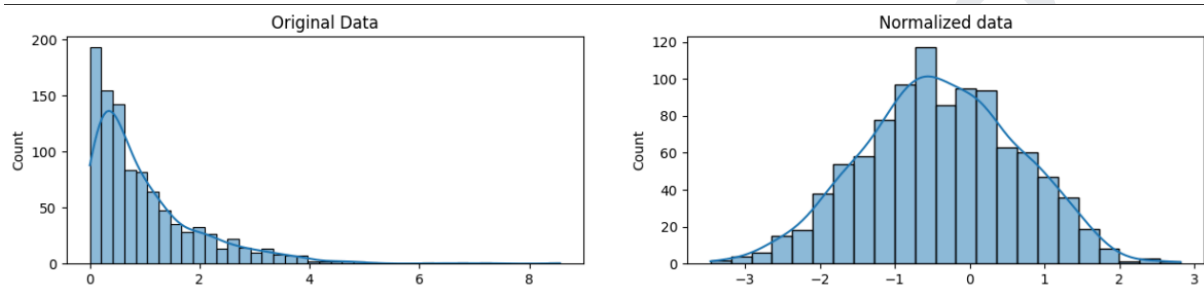
Scaling and Normalization

```
import pandas as pd
import numpy as np
from scipy import stats
from mlxtend.preprocessing import minmax_scaling
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(0)

original_data = np.random.exponential(size=1000)
scaled_data = minmax_scaling(original_data, columns=[0])
fig, ax = plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(scaled_data, ax=ax[1], kde=True, legend=False)
ax[1].set_title("Scaled data")
plt.show()
```



```
normalized_data = stats.boxcox(original_data)
fig, ax=plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(normalized_data[0], ax=ax[1], kde=True, legend=False)
ax[1].set_title("Normalized data")
plt.show()
```



Parsing Dates

```
import pandas as pd
import numpy as np
import seaborn as sns
import datetime

landslides = pd.read_csv("../input/landslide-events/catalog.csv")
np.random.seed(0)
```

```
landslides.head()
```

	id	date	time	continent_code	country_name	country_code	state/province	population	city/town	distance	...	geolocation	hazard_type	landslide_type	landslide_size	trigger	stop
0	34	3/2/07	Night	NaN	United States	US	Virginia	16000	Cherry Hill	3.40765	...	(38.600900000000003, -77.268199999999993)	Landslide	Landslide	Small	Rain	
1	42	3/22/07	NaN	NaN	United States	US	Ohio	17288	New Philadelphia	3.33522	...	(40.517499999999998, -81.430499999999995)	Landslide	Landslide	Small	Rain	
2	56	4/6/07	NaN	NaN	United States	US	Pennsylvania	15930	Wilksburg	2.91977	...	(40.4377, -79.915999999999997)	Landslide	Landslide	Small	Rain	
3	59	4/14/07	NaN	NaN	Canada	CA	Quebec	42786	Châteauguay	2.98682	...	(45.322600000000001, -73.777100000000004)	Landslide	Riverbank collapse	Small	Rain	
4	61	4/15/07	NaN	NaN	United States	US	Kentucky	6903	Pikeville	5.66542	...	(37.432499999999997, -82.493099999999996)	Landslide	Landslide	Small	Downpour	

```
print(landslides['date'].head())
```

```
0    3/2/07
1    3/22/07
2    4/6/07
3    4/14/07
4    4/15/07
Name: date, dtype: object
```

```
landslides['date'].dtype
```

```
dtype('O')
```

```
landslides['date_parsed'] = pd.to_datetime(landslides['date'], format="%m/%d/%y")
landslides['date_parsed'].head()
```

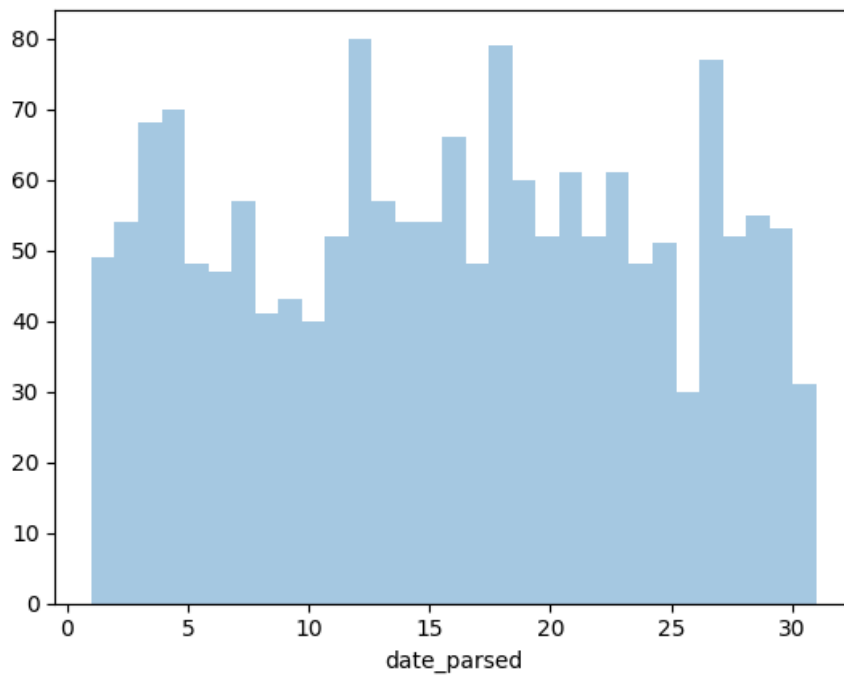
```
date_parsed
0    2007-03-02
1    2007-03-22
2    2007-04-06
3    2007-04-14
4    2007-04-15
dtype: datetime64[ns]
```

```
day_of_month_landslides = landslides['date_parsed'].dt.day
day_of_month_landslides.head()
```

```
date_parsed
0         2.0
1        22.0
2         6.0
3        14.0
4        15.0
```



```
day_of_month_landslides = day_of_month_landslides.dropna()  
sns.distplot(day_of_month_landslides, kde=False, bins=31)
```



Character Encodings

```
before = "This is the euro symbol: €"
```

```
type(before)
```

```
str
```

```
after = before.encode("utf-8", errors="replace")
```

```
type(after)
```

```
bytes
```

```
after
```

```
b'This is the euro symbol: \xe2\x82\xac'
```

```
print(after.decode("utf-8"))
```

```
This is the euro symbol: €
```

```
print(after.decode("ascii"))
```

```
-  
UnicodeDecodeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_2210/938697088.py in <cell line: 0>()  
----> 1 print(after.decode("ascii"))  
  
UnicodeDecodeError: 'ascii' codec can't decode byte 0xe2 in position 25:  
ordinal not in range(128)
```

```
before = "This is the euro symbol: €"
after = before.encode("ascii", errors = "replace")
print(after.decode("ascii"))
```

```
This is the euro symbol: ?
```

```
kickstarter_2016 = pd.read_csv("../input/kickstarter-projects/ks-projects-201612.csv")
```

```
/tmp/ipykernel_2210/3779710350.py:1: DtypeWarning: Columns (14,15) have mixed types. Specify dtype option on import or set low_memory
kickstarter_2016 = pd.read_csv("../content/ks-projects-201801-utf8.csv")
```

```
with open("../input/kickstarter-projects/ks-projects-201801.csv", 'rb') as rawdata:
    result = charset_normalizer.detect(rawdata.read(10000))
print(result)
```

```
{'encoding': 'utf-8', 'language': 'Spanish', 'confidence': 0.991}
```

```
kickstarter_2016 = pd.read_csv("../input/kickstarter-projects/ks-projects-201612.csv",
encoding='Windows-1252')
kickstarter_2016.head()
```

Inconsistent Data Entry

```
import pandas as pd
import numpy as np
import fuzzywuzzy
from fuzzywuzzy import process
import charset_normalizer

professors = pd.read_csv("../input/pakistan-intellectual-capital/pakistan_intellectual_capital.csv")
np.random.seed(0)
```

```
professors.head()
```

Unnamed: 0	S#	Teacher Name	University Currently Teaching	Department	Province University Located	Designation	Terminal Degree	Graduated from	Country	Year	Specialization/Research Interests	Other Information	
0	2	3	Dr. Abdul Basit	University of Balochistan	Computer Science & IT	Balochistan	Assistant Professor	PhD	Asian Institute of Technology	Thailand	NaN	Software Engineering & DBMS	NaN
1	4	5	Dr. Waheed Noor	University of Balochistan	Computer Science & IT	Balochistan	Assistant Professor	PhD	Asian Institute of Technology	Thailand	NaN	DBMS	NaN
2	5	6	Dr. Junaid Baber	University of Balochistan	Computer Science & IT	Balochistan	Assistant Professor	PhD	Asian Institute of Technology	Thailand	NaN	Information processing, Multimedia mining	NaN
3	6	7	Dr. Maheen Bakhtyar	University of Balochistan	Computer Science & IT	Balochistan	Assistant Professor	PhD	Asian Institute of Technology	Thailand	NaN	NLP, Information Retrieval, Question Answering...	NaN
4	24	25	Samina Azim	Sardar Bahadur Khan Women's University	Computer Science	Balochistan	Lecturer	BS	Balochistan University of Information Technolo...	Pakistan	2005.0	VLSI Electronics DLD Database	NaN

```
countries = professors['Country'].unique()
```

```
countries.sort()
```

```
countries
```

```
array([' Germany', ' New Zealand', ' Sweden', ' USA', 'Australia',
      'Austria', 'Canada', 'China', 'Finland', 'France', 'Greece',
      'HongKong', 'Ireland', 'Italy', 'Japan', 'Macau', 'Malaysia',
      'Mauritius', 'Netherland', 'New Zealand', 'Norway', 'Pakistan',
      'Portugal', 'Russian Federation', 'Saudi Arabia', 'Scotland',
      'Singapore', 'South Korea', 'SouthKorea', 'Spain', 'Sweden',
      'Thailand', 'Turkey', 'UK', 'USA', 'USofA', 'Urbana', 'germany'],
      dtype=object)
```

```
professors['Country'] = professors['Country'].str.lower()
```

```
professors['Country'] = professors['Country'].str.strip()
```

```
countries = professors['Country'].unique()
```

```
countries.sort()
```

```
countries
```

```
array(['australia', 'austria', 'canada', 'china', 'finland', 'france',  
      'germany', 'greece', 'hongkong', 'ireland', 'italy', 'japan',  
      'macau', 'malaysia', 'mauritius', 'netherland', 'new zealand',  
      'norway', 'pakistan', 'portugal', 'russian federation',  
      'saudi arabia', 'scotland', 'singapore', 'south korea',  
      'southkorea', 'spain', 'sweden', 'thailand', 'turkey', 'uk',  
      'urbana', 'usa', 'usofa'], dtype=object)
```

```
matches = fuzzywuzzy.process.extract("south korea", countries, limit=10,  
scorer=fuzzywuzzy.fuzz.token_sort_ratio)
```

```
matches
```

```
[('south korea', 100),  
 ('southkorea', 48),  
 ('saudi arabia', 43),  
 ('norway', 35),  
 ('austria', 33),  
 ('ireland', 33),  
 ('pakistan', 32),  
 ('portugal', 32),  
 ('scotland', 32),  
 ('australia', 30)]
```

```
def replace_matches_in_column(df, column, string_to_match, min_ratio = 47):
```

```
strings = df[column].unique() matches =
```

```
fuzzywuzzy.process.extract(string_to_match, strings,
```

```
limit=10, scorer=fuzzywuzzy.fuzz.token_sort_ratio)
```

```
close_matches = [matches[0] for matches in matches if matches[1] >= min_ratio]
```

```
rows_with_matches = df[column].isin(close_matches)
df.loc[rows_with_matches, column] = string_to_match
print("All done!")
```

```
replace_matches_in_column(df=professors,column='Country',
string_to_match="south korea")
```

```
All done!
```

```
countries = professors['Country'].unique()
countries.sort()
countries
```

```
array(['australia', 'austria', 'canada', 'china', 'finland', 'france',
      'germany', 'greece', 'hongkong', 'ireland', 'italy', 'japan',
      'macau', 'malaysia', 'mauritius', 'netherland', 'new zealand',
      'norway', 'pakistan', 'portugal', 'russian federation',
      'saudi arabia', 'scotland', 'singapore', 'south korea',
      'southkorea', 'spain', 'sweden', 'thailand', 'turkey', 'uk',
      'urbana', 'usa', 'usofa'], dtype=object)
```